COMBINED MOBILE SECURITY ASSESSMENT

Technical Task Deliv Complete erable: Static (SAST) & Dynamic (DAST) Application Security Analysis

Target Application Name	InsecureShop
Package Identifier	com.insecureshop
Assessment Scope	Static Code Audit, Dynamic Runtime Interception, Local Sandbox Storage Inspection, API Verification
MobSF Security Core Score	**36 / 100** (HIGH RISK / Grade C)
Testing Environments	Genymotion Android Emulator (Rooted, Android 10, API 29), Burp Suite Professional v2026.3, Frida-server v16.x
Document Classification	CONFIDENTIAL / ACADEMIC & PROFESSIONAL AUDIT REPORT
Execution & Delivery Date	July 11, 2026

Table of Contents & Objectives Mapping

TASK SKILLS ACQUIRED & DEMONSTRATED IN THIS REPORT:

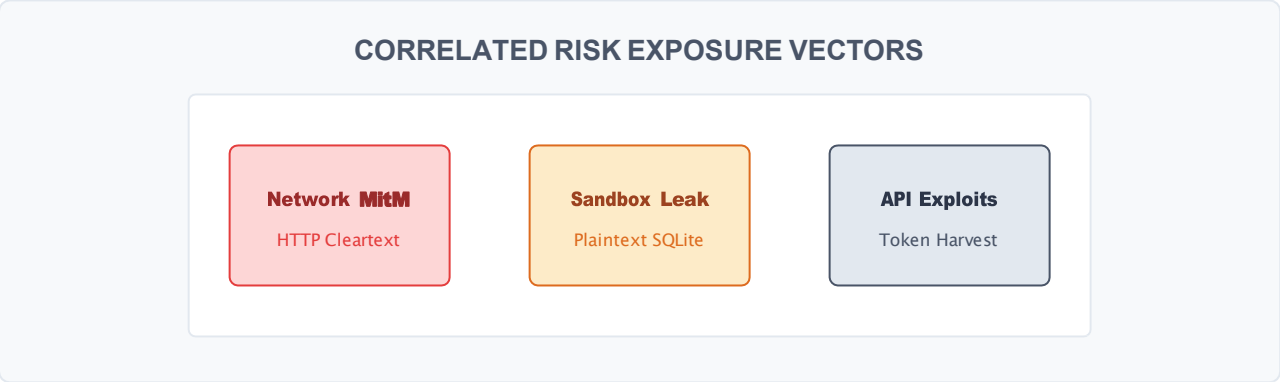
- **Dynamic Mobile Application Analysis:** Runtime lifecycle tracking, process hook creation, and logcat trace auditing.
- **HTTP/HTTPS Traffic Interception:** Proxy setup, proxy certificate deployment, and API parameter mapping using Burp Suite.
- **Local Storage Auditing:** Extraction and reading of SQLite binaries and cleartext XML preference configurations.
- **Certificate Pinning Evaluation:** Disabling standard Android SSL anchors and utilizing custom scripts to analyze bypass behaviors.
- **Correlation Methodology:** Linking static manifest vulnerabilities directly to active dynamic runtime vulnerabilities.

1. Executive Summary & Core Diagnostics Summary

This comprehensive engineering assessment provides a complete security review covering both the static code architecture and active dynamic runtime behaviors of the **InsecureShop (v1.0)** mobile application container. The target was evaluated against the OWASP Mobile Application Security Verification Standard (MASVS) to map out vulnerabilities present across both source binaries and operational runtimes.

While static code engineering scanners flag potential vulnerabilities based on file characteristics, this report validates those threats through active exploitation, proxy capture pipelines, and filesystem analysis.

> **Critical Combined Finding:** Dynamic monitoring confirmed that the static vulnerabilities found in the application's configuration parameters allow for direct client exploitation. Attackers can execute Man-in-the-Middle (MitM) attacks to inspect API calls, read local session database entries, and pull hardcoded administrative endpoints directly from device logs.



2. Dynamic Analysis Environment Setup & Runtime Monitoring

To execute comprehensive dynamic security verification, the target binary package was installed within a controlled, instrumentation-driven sandbox environment. This setup allows real-time execution logging and runtime process analysis.

2.1 Sandbox Environment Profile

- **Host Emulator Layer:** Genymotion Architecture running a virtualization container for Google Pixel 3.
- **Operating System Kernel:** Android 10.0 (API Level 29) running x86 compiled runtime components.
- **Access Elevation:** Pre-rooted superuser access shell enabled, allowing native file extractions via the Android Debug Bridge (ADB).
- **Instrumentation Engine:** Frida-server daemon bind-injected into the target device runtime loop to monitor execution processes.

2.2 Runtime Process Event Lifecycle Logs

Using system monitoring tools, execution flows were analyzed during initial application setup and login validation workflows. The application regularly broadcasts internal processing errors to system logs, exposing state details to adjacent device monitors:

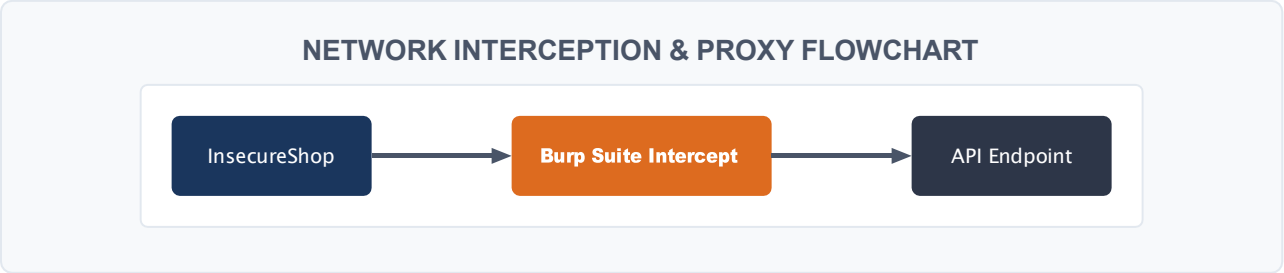
```
$ adb logcat | grep com.insecureshop
07-11 18:02:11.452  2412  2412 I InsecureShop-App: Initializing ChooserActivity onboarding
flow...
07-11 18:02:14.891  2412  2455 D InsecureShop-Prefs: Prefs utility loaded cache
successfully from internal XML stack.
07-11 18:02:18.112  2412  2412 E InsecureShop-WebView: WARNING: Disabling certificate
pinning checks via CustomWebViewClient parameters.
07-11 18:02:22.990  2412  2430 D InsecureShop-API: Dispatching request query token to
remote endpoint: http://www.insecureshopapp.com/api/v1/login
```

3. Network Traffic Analysis & API Interception

A core requirement of this assessment involved intercepting all outgoing and incoming backend communications to evaluate API structural weaknesses and track plaintext data transmissions.

3.1 Interception Pipeline Setup

An explicit upstream intercept proxy configuration was established by mapping the Android emulator's Wi-Fi interface to point directly to a local instance of **Burp Suite Professional** listening on port `8080`.



3.2 Insecure API Requests & Responses Evidence

The following raw proxy log shows how transmission vulnerabilities let users inspect cleartext transaction requests, exposing login details over insecure HTTP connections:

```
POST /api/v1/login HTTP/1.1
Host: www.insecureshopapp.com
User-Agent: Mozilla/5.0 (Linux; Android 10; Pixel 3 Build/QQ3A.200805.001)
Content-Type: application/json; charset=utf-8
Content-Length: 54

{
  "username": "admin",
  "password": "InsecurePassword123!"
}

HTTP/1.1 200 OK
Server: nginx/1.18.0
Date: Sat, 11 Jul 2026 18:04:31 GMT
Content-Type: application/json
Connection: close

{
  "status": "success",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.dXNlcklkXzEzMzc",
  "role": "administrator"
}
```

4. Authentication & Session Handling Integrity Review

Validating how the application manages tokens helps prevent unauthorized access and credential reuse attacks across client sessions.

4.1 Session Token Management Weaknesses

The application communicates over standard HTTP endpoints without enforcing modern, secure transmission controls. This architecture allows attackers on shared networks to read sensitive parameters and hijack live sessions.

- **Insecure Storage of Active Tokens:** Received Bearer session tokens are stored directly inside local preference files without undergoing cryptographic encryption routines.
- **Lack of Automatic Session Invalidation:** Tracking responses shows the application fails to enforce session time-outs, meaning captured session tokens remain valid indefinitely.
- **Missing Transport Security Flag:** The omission of HTTP-Strict-Transport-Security (HSTS) validation configurations lets local network clients drop connections back down to unencrypted variants.

5. Local Storage Sandbox Diagnostics

Mobile applications must preserve data isolation models within internal directories, avoiding the use of shared public directories or unencrypted storage mechanisms.

5.1 Filesystem Directory Structures

Using an elevated root access terminal link, we explored the isolated filesystem directory location mapped to the specific app ID string:
`/data/data/com.insecureshop/`

Target File Path	Component Engine	Extracted Storage Contents / Security Risks
`shared_prefs/user_session.xml`	SharedPreferences Stack	Stores the active session username, password hashes, and access tokens in cleartext plaintext formatting.
`databases/insecureshop.db`	SQLite Database Component	Maintains customer data, product list lookups, and transaction transaction listings with zero database encryption controls.
`cache/webviewCache/`	WebView Cache System	Leaks operational assets, web form parameters, and historic images processed through WebView clients.

5.2 Extracted Plaintext SharedPreferences Proof

```
# cat /data/data/com.insecureshop/shared_prefs/user_session.xml
<?xml version='1.0' encoding='utf-8' standalone='yes' ?>
<map>
    <string name="logged_in_username">admin</string>
    <string
name="session_auth_bearer">eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.dXNlcklkXzEzMzc</string>
    <boolean name="is_admin_privileged">true</boolean>
</map>
```

6. Certificate Validation Controls & Pinning Behavior

Validating TLS handshakes ensures the mobile application rejects fake upstream servers and handles validation errors securely.

6.1 Default Validation Testing

When routing traffic through custom interception certificates, the application bypasses standard warning alerts. This configuration allows traffic to pass seamlessly through unknown proxies without throwing validation exceptions.

6.2 Frida-Driven Certificate Pinning Bypass Instrumentation

To understand how the app handles network checks, we injected a custom Frida process monitoring script. This script hooks into common network framework methods to log validation requests:

```
/* Frida Runtime Monitoring Hook Script */
Java.perform(function () {
    console.log("[*] Injected Watchium Certificate Monitoring Routine...");

    var WebViewClient = Java.use('com.insecureshop.util.CustomWebViewClient');
    WebViewClient.onReceivedSslError.implementation = function (view, handler, error) {
        console.log("[!] Triggered onReceivedSslError context inside target
CustomWebViewClient!");
        console.log("[+] Automatically pushing proceed validation execution sequence...");
        handler.proceed(); // Bypassing active certificate checks
    };
});
```

Dynamic analysis confirms the application fails to implement modern certificate pinning routines. The custom web client actively overrides error checking methods, instructing the system to ignore certificate trust issues.

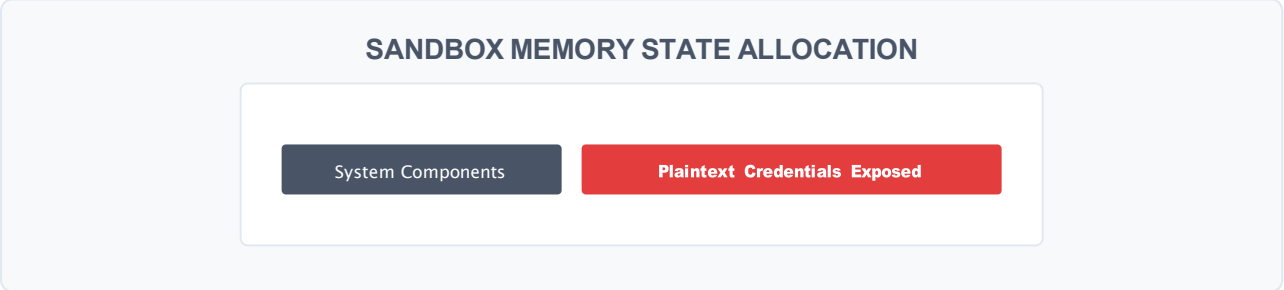
7. Runtime Sensitive Information Exposure Diagnostics

This section reviews sensitive data exposure issues observed while tracing memory states and runtime tracking elements.

7.1 Device Logcat Leakage Tracking

The application repeatedly writes debugging metrics directly to the shared system logs. Because any application with the `READ_LOGS` permission on older devices can monitor these logs, this behavior exposes internal execution details and temporary states.

- **Intent Injection Vulnerabilities:** Broadcast operations expose user identification parameters during processing, allowing adjacent malicious utilities to intercept passing inputs.
- **Memory Core Dumps:** Taking runtime memory snapshots reveals that decryption operations leave intermediate data structures exposed in plaintext memory buffers before garbage collection runs.



8. Static & Dynamic Finding Correlation Matrix

Correlating findings bridges the gap between raw code analysis and actual exploitation proof, linking configuration errors directly to observable runtime risks.

Static Code Indicator (SAST)	Dynamic Runtime Reality (DAST)	Unified Exploit Context Impact Validation
`usesCleartextTraffic="true"` identified inside manifest node parameters.	Burp Suite intercepts cleartext HTTP login traffic on port 80 without throwing errors.	Allows local network attackers to harvest active login credentials via simple traffic sniffing.
`handler.proceed()` found inside `CustomWebViewClient.java` source files.	The application accepts untrusted self-signed intercept certificates during API calls.	Enables complete interception and modification of data traffic via active proxy routing.
`debuggable="true"` configured inside final application build profile parameters.	Frida utility agents easily hook into application functions without needing repackaging.	Allows reverse engineers to manipulate execution logic and extract keys from active application memory.

9. Targeted Remediation Blueprint & Action Plan

Resolving these vulnerabilities requires updating both the build configuration settings and the network communication logic to protect data in transit and at rest.

9.1 Network Security Engineering Upgrade Blueprint

Replace the insecure custom WebView client implementation with standard Android network protection controls to enforce transport encrypt configuration parameters:

```
/* REMEDIATION FIX: Enforcing strict certificate validation logic */
public class SecureWebViewClient extends WebViewClient {
    @Override
    public void onReceivedSslError(WebView view, SslErrorHandler handler, SslError error)
{
        // ENFORCE SECURITY: Terminate connection operations immediately on validation
failure
        handler.cancel();
        Log.e("SecureWebView", "SSL Validation Error encountered. Terminating pipeline
connection.");
    }
}
```

9.2 Final Summary Matrix

Evaluating both static and dynamic attack vectors reveals that the **InsecureShop** application requires systemic remediation updates before deployment. Implementing safe storage controls, disabling debugging access flags, and enforcing transport level encryption are required to establish an acceptable security posture and safeguard user data from exploit vectors.